

CLIP: Symmetric Contrastive Loss.

CLIP trains an image encoder and a text encoder *together* so that a picture and its caption land at the same spot on a shared sphere. The trick is a single batch-wide classification game played in two directions at once — match each image to its caption, and each caption to its image — sharpened by one learned knob, the temperature. Every number here — the 4×4 similarity matrix, both cross-entropies, and the averaged loss — is computed in full, nothing summarised or deferred.

THE EQUATION

$$\mathcal{L} = \frac{1}{2}(\mathcal{L}_{\text{img}} + \mathcal{L}_{\text{txt}}), \quad \mathbf{L} = (\hat{\mathbf{I}} \hat{\mathbf{T}}^\top) \cdot e^t$$

Worked example. A batch of $N = 4$ image and text embeddings in $\mathbb{R}^3$; L2-normalise, build the 4×4 cosine matrix, scale by $e^t = 14.3$ (CLIP's init $1/0.07$), softmax both directions, and average the two cross-entropies.

Topic 1 of 1 in this module · companion to the course page · v1.0

Contents

1	The claim: one batch, two classification problems	2
2	The data-flow (TikZ)	3
3	The derivation: both directions	3
4	Worked example: $N = 4$ pairs in $\mathbb{R}^3$	4
5	Zero-shot classification: the payoff	6
6	Properties / consequences that matter	6
7	Where this sits in the track	7
A	Reproducible (NumPy)	7

1 The claim: one batch, two classification problems

CLIP (Contrastive Language–Image Pre-training) feeds a batch of N matched (image, text) pairs through two separate encoders. Let $\hat{\mathbf{I}} \in \mathbb{R}^{N \times d}$ and $\hat{\mathbf{T}} \in \mathbb{R}^{N \times d}$ hold the N image and text embeddings, each row $L2$ -normalised. The model scores every image against every text, scaling the cosine similarities by a learned temperature, and asks each image to pick out its own caption (and vice-versa):

$$\mathbf{L} = (\hat{\mathbf{I}} \hat{\mathbf{T}}^\top) e^t, \quad \mathcal{L} = \frac{1}{2}(\mathcal{L}_{\text{img}} + \mathcal{L}_{\text{txt}}), \quad \mathcal{L}_{\text{img}} = \frac{1}{N} \sum_i \text{CE}(\mathbf{L}_{i,:}, i) \quad (1)$$

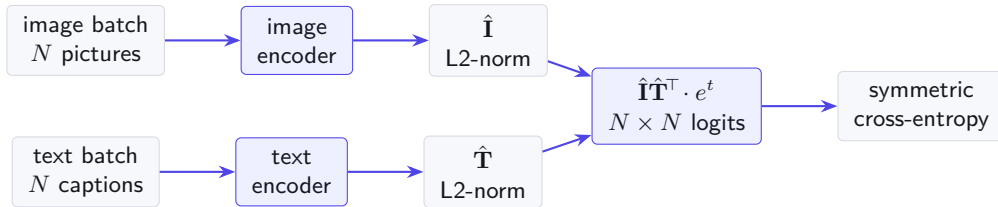
The matrix $\mathbf{L} \in \mathbb{R}^{N \times N}$ is the table of *logits*. The *correct* pairs sit on the diagonal: $\mathbf{L}_{ii}$ scores image i against text i . Off-diagonal entries are *in-batch negatives* — mismatched image/text pairs that the loss should push down.

Symbol	Meaning	Shape
$\hat{\mathbf{I}}$	L2-normalised image embeddings (rows)	$\mathbb{R}^{N \times d}$
$\hat{\mathbf{T}}$	L2-normalised text embeddings (rows)	$\mathbb{R}^{N \times d}$
$\hat{\mathbf{I}}\hat{\mathbf{T}}^\top$	cosine-similarity matrix, entries in $[-1, 1]$	$\mathbb{R}^{N \times N}$
t	learned log-temperature (clamped)	scalar
e^t	temperature scale (logit multiplier)	scalar > 0
$\mathbf{L}$	scaled logits	$\mathbb{R}^{N \times N}$
$\text{CE}(\cdot, i)$	cross-entropy, target = class i	scalar ≥ 0
$\mathcal{L}$	symmetric contrastive loss	scalar ≥ 0

Picture an $N \times N$ grid where row i is an image and column j is a caption. The cell (i, j) glows according to how aligned they are. Training is a seating game: every image must learn to glow brightest at its own caption's column, and every caption must glow brightest at its own image's row. When the diagonal lights up and everything else goes dark, the two encoders have agreed on a shared meaning-space.

2 The data-flow (TikZ)

Two encoders, one shared sphere, one matrix of dot products:



The only place the two streams meet is the matrix multiply — which is why the loss must be *symmetric*: gradients flow back into both encoders through the same $N \times N$ block.

3 The derivation: both directions

A softmax over a row of $\mathbf{L}$ asks “given image i , which text is it?”. With the true class being i (the diagonal), the per-row cross-entropy is the negative log-probability of the diagonal entry:

$$\mathcal{L}_{\text{img}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{e^{\mathbf{L}_{ii}}}{\sum_{k=1}^N e^{\mathbf{L}_{ik}}} \quad (2)$$

The text direction is the *same matrix read down its columns*: “given text j , which image is it?”. Softmax over column j (i.e. over the rows $\mathbf{L}_{kj}$), true class j :

$$\mathcal{L}_{\text{txt}} = -\frac{1}{N} \sum_{j=1}^N \log \frac{e^{\mathbf{L}_{jj}}}{\sum_{k=1}^N e^{\mathbf{L}_{kj}}} \quad (3)$$

Both share the diagonal in the numerator but normalise over a different axis (row vs. column), so they are genuinely two losses. The final objective averages them: $\mathcal{L} = \frac{1}{2}(\mathcal{L}_{\text{img}} + \mathcal{L}_{\text{txt}})$.

Why the temperature scales the logits. The cosine matrix lives in $[-1, 1]$ — far too flat for a confident softmax. Multiplying by e^t stretches the gaps before the softmax. This is exactly the scaling lever from Module 0.1 (dividing attention by $\sqrt{d_k}$ controls logit sharpness) and the sampling temperature of Module 2.1: a *low* temperature $\tau = e^{-t}$ (large e^t) gives *sharper* probabilities. CLIP stores $t = \log(1/\tau)$ and learns it, initialised at $\tau = 0.07$, i.e. $e^t = 1/0.07 \approx 14.3$.

4 Worked example: $N = 4$ pairs in $\mathbb{R}^3$

Take a batch of four image embeddings and four text embeddings (rows):

$$\mathbf{I} = \begin{bmatrix} 2 & 1 & 2 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \\ 3 & 0 & 1 \end{bmatrix}, \quad \mathbf{T} = \begin{bmatrix} 3 & 1 & 2 \\ 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 0 & 1 \end{bmatrix}.$$

4.1 Step 1 — L2-normalise every row

Row norms: $\|I_1\| = \sqrt{9} = 3$, $\|I_2\| = \sqrt{11} \approx 3.3166$, $\|I_3\| = \sqrt{5} \approx 2.2361$, $\|I_4\| = \sqrt{10} \approx 3.1623$; and $\|T_1\| = \sqrt{14} \approx 3.7417$, $\|T_2\| = \sqrt{5} \approx 2.2361$, $\|T_3\| = \sqrt{10} \approx 3.1623$, $\|T_4\| = \sqrt{5} \approx 2.2361$. Dividing each row by its norm:

$$\hat{\mathbf{I}} = \begin{bmatrix} 0.6667 & 0.3333 & 0.6667 \\ 0.3015 & 0.9045 & 0.3015 \\ 0 & 0.4472 & 0.8944 \\ 0.9487 & 0 & 0.3162 \end{bmatrix}, \quad \hat{\mathbf{T}} = \begin{bmatrix} 0.8018 & 0.2673 & 0.5345 \\ 0.4472 & 0.8944 & 0 \\ 0 & 0.3162 & 0.9487 \\ 0.8944 & 0 & 0.4472 \end{bmatrix}.$$

4.2 Step 2 — cosine matrix $\mathbf{S} = \hat{\mathbf{I}}\hat{\mathbf{T}}^\top$

Each entry $\mathbf{S}_{ij} = \hat{I}_i \cdot \hat{T}_j$ is a dot product of unit vectors. For instance $\mathbf{S}_{11} = 0.6667(0.8018) + 0.3333(0.2673) + 0.6667(0.5345) = 0.9800$ and $\mathbf{S}_{14} = 0.6667(0.8944) + 0 + 0.6667(0.4472) = 0.8944$. The full matrix:

$$\mathbf{S} = \begin{bmatrix} \mathbf{0.9800} & 0.5963 & 0.7379 & 0.8944 \\ 0.6447 & \mathbf{0.9439} & 0.5721 & 0.4045 \\ 0.5976 & 0.4000 & \mathbf{0.9900} & 0.4000 \\ 0.9297 & 0.4243 & 0.3000 & \mathbf{0.9900} \end{bmatrix}.$$

The diagonal (true pairs, bold) is the largest entry in every row and every column — a clean signal. Heatmap of $\mathbf{S}$ (intensity = $\lfloor \cos \times 80 \rfloor$, so $1 \rightarrow \text{indigo!80}$; the bright diagonal is the correct matches):

	T_1	T_2	T_3	T_4
I_1	0.980	0.596	0.738	0.894
I_2	0.645	0.944	0.572	0.405
I_3	0.598	0.400	0.990	0.400
I_4	0.930	0.424	0.300	0.990

4.3 Step 3 — scale by the temperature $e^t = 14.3$

Multiply every entry by 14.3 to get the logits $\mathbf{L} = 14.3 \mathbf{S}$:

$$\mathbf{L} = \begin{bmatrix} 14.013 & 8.527 & 10.551 & 12.790 \\ 9.219 & 13.497 & 8.181 & 5.785 \\ 8.546 & 5.720 & 14.156 & 5.720 \\ 13.294 & 6.067 & 4.290 & 14.156 \end{bmatrix}.$$

The $14.3\times$ amplification turns a cosine gap of 0.98 vs. 0.89 (row 1) into a logit gap of 14.01 vs. 12.79 — now large enough for the softmax to commit.

4.4 Step 4 — image→text softmax (rows, Eq. 2)

Softmax each row of $\mathbf{L}$. Take row 1: subtracting the max (14.013) and exponentiating gives $[1, 0.00414, 0.03137, 0.29432]$ summing to 1.3298, so $\text{softmax}(\mathbf{L}_{1,:}) = [0.7520, 0.0031, 0.0236, 0.2213]$. Doing all four rows:

$$\mathbf{P}^{\text{row}} = \begin{bmatrix} \mathbf{0.7520} & 0.0031 & 0.0236 & 0.2213 \\ 0.0136 & \mathbf{0.9812} & 0.0048 & 0.0004 \\ 0.0036 & 0.0002 & \mathbf{0.9959} & 0.0002 \\ 0.2969 & 0.0002 & 0.0000 & \mathbf{0.7029} \end{bmatrix}.$$

The diagonal probabilities are $[0.7520, 0.9812, 0.9959, 0.7029]$. The per-row cross-entropies $-\log \mathbf{P}_{ii}^{\text{row}}$ are $[0.2851, 0.0190, 0.0041, 0.3525]$, so

$$\mathcal{L}_{\text{img}} = \frac{1}{4}(0.2851 + 0.0190 + 0.0041 + 0.3525) = 0.16518.$$

Rows 1 and 4 carry almost all the loss: image 1 is nearly as similar to text 4 (0.894) as to its own (0.980), and image 4 to text 1 — the confusable pairs the gradient will work hardest on.

4.5 Step 5 — text→image softmax (columns, Eq. 3)

Now softmax *down each column* of the same $\mathbf{L}$. Reading the columns as rows:

$$\mathbf{P}^{\text{col}} = \begin{bmatrix} \mathbf{0.6668} & 0.0055 & 0.0028 & 0.3249 \\ 0.0069 & \mathbf{0.9921} & 0.0004 & 0.0006 \\ 0.0264 & 0.0025 & \mathbf{0.9711} & 0.0001 \\ 0.2032 & 0.0002 & 0.0002 & \mathbf{0.7964} \end{bmatrix},$$

where row j here is the softmax over column j of $\mathbf{L}$. The diagonal gives $-\log$ values $[0.4053, 0.0079, 0.0294, 0.2276]$ hence

$$\mathcal{L}_{\text{txt}} = \frac{1}{4}(0.4053 + 0.0079 + 0.0294 + 0.2276) = 0.16753.$$

4.6 Step 6 — average the two directions (Eq. 1)

$$\mathcal{L} = \frac{1}{2}(\mathcal{L}_{\text{img}} + \mathcal{L}_{\text{txt}}) = \frac{1}{2}(0.16518 + 0.16753) = \boxed{0.16635}. \quad (4)$$

Because the diagonal is the argmax of every row *and* every column, a zero-shot prediction (argmax over a row) is already correct for all four pairs; the residual loss is the price of the not-yet-saturated confusable off-diagonals.

The whole objective is one batch-wide N -way classification, run twice on the *same* logit matrix: once across rows (image→text) and once down columns (text→image). The diagonal is the label in both. Averaging the two cross-entropies forces a *symmetric* agreement between the encoders, and the learned temperature e^t is the single dial that sets how confident the softmax is allowed to be.

5 Zero-shot classification: the payoff

At test time there is no diagonal — there is one image and a list of *candidate* class names. Wrap each class in a prompt, “a photo of a {class}”, encode all prompts with the text encoder, encode the image, L2-normalise, and pick the prompt with the highest cosine similarity:

$$\hat{y} = \arg \max_c \hat{I} \cdot \hat{T}_c.$$

This is exactly Step 2 applied to one image row against a column of class-prompt embeddings — the temperature cancels in the argmax, so no training on the target classes is needed. In our worked batch, taking each image’s row and reading off the argmax recovers $[T_1, T_2, T_3, T_4]$: perfect retrieval.

6 Properties / consequences that matter

1. **Lower bound 0.** Both $\mathcal{L}_{\text{img}}, \mathcal{L}_{\text{txt}} \geq 0$ and so is their average; $\mathcal{L} \rightarrow 0$ only as every diagonal softmax probability $\rightarrow 1$.
2. **$N - 1$ free negatives per example.** Each row pits the true caption against the other $N - 1$ captions in the batch; no negative mining or extra labels are required.
3. **Symmetry matters.** Using only $\mathcal{L}_{\text{img}}$ would let the text encoder drift; averaging both directions ties the two encoders to one shared geometry.
4. **Temperature is learned, not fixed.** Storing $t = \log(1/\tau)$ and optimising it lets the model anneal its own sharpness; it enters the loss linearly as the logit scale e^t .
5. **Scale invariance.** L2-normalising the embeddings makes the logits depend only on *angles*, so a long caption cannot win on magnitude (the same reasoning as Module 1.2’s cosine).

Two failure modes. First, **small batches starve the loss**: with $N = 4$ there are only 3 negatives per row, so the task is easy and the embeddings separate poorly — CLIP used

$N \approx 32,768$ precisely to make the softmax hard. Second, **an unclamped temperature collapses training**: if e^t is free to grow, the model can drive logits to $\pm\infty$ and the softmax saturates, killing gradients (or exploding them). CLIP clamps t so that $e^t \leq 100$ (equivalently $\tau \geq 0.01$). And, as always, forgetting to L2-normalise turns the cosine matrix back into raw dot products that conflate “aligned” with “large”.

7 Where this sits in the track

This is the loss at the heart of every contrastive vision-language model. It generalises Module 1.2’s single-anchor InfoNCE to a full *batch matrix* and runs it symmetrically; it reuses the temperature-scaling intuition of Module 0.1 (attention $\sqrt{d_k}$) and Module 2.1 (sampling temperature). Downstream, the aligned image/text space powers zero-shot classification, retrieval, and the frozen-vision-encoder inputs to multimodal LLMs — the rest of Phase 6.

A Reproducible (NumPy)

Inputs: image rows **I** and text rows **T** as above, temperature $e^t = 14.3$. The snippet regenerates the cosine matrix, both softmaxes, and the averaged loss.

```
import numpy as np
I = np.array([[2.,1.,2.],[1.,3.,1.],[0.,1.,2.],[3.,0.,1.]])
T = np.array([[3.,1.,2.],[1.,2.,0.],[0.,1.,3.],[2.,0.,1.]])
In = I/np.linalg.norm(I,axis=1,keepdims=True)
Tn = T/np.linalg.norm(T,axis=1,keepdims=True)
L = (In @ Tn.T) * 14.3 # scaled logits, N x N
sm = lambda X: np.exp(X-X.max(1,keepdims=True)) / \
            np.exp(X-X.max(1,keepdims=True)).sum(1,keepdims=True)
Prow, Pcol = sm(L), sm(L.T) # image->text rows, text->image cols
Limg = -np.log(np.diag(Prow)).mean()
Ltxt = -np.log(np.diag(Pcol)).mean()
print(np.round(In@Tn.T,4)) # cosine matrix; diagonal is the row/col max
print(round(Limg,5), round(Ltxt,5)) # 0.16518 0.16753
print(round(0.5*(Limg+Ltxt),5)) # 0.16635
print((In@Tn.T).argmax(1)) # [0 1 2 3] zero-shot is correct
```